Implementation Plan — Betweenness Centrality Prediction Inductive Betweenness Centrality Prediction on Synthetic Graph Families

Contents

1 Research question and scope **2**
- 1.1 Central question . . . 2
- 1.2 What "success" means . . . 2
- 1.3 Inductive-only experimental protocol . . . 2

2 Main hypotheses **2**

3 Task definition: labels, objectives, metrics **2**
- 3.1 Datasets (inductive protocol) . . . 2
- 3.2 Betweenness centrality labels . . . 2
- 3.3 Target transformation . . . 3
- 3.4 Training objectives . . . 3
- 3.5 Evaluation metrics . . . 3

4 Inputs: features and structural encodings **3**
- 4.1 Feature sets . . . 3

5 Models to compare **3**
- 5.1 Baselines . . . 3
- 5.2 Proposed: multi-stream routing in GNNs . . . 4
  - 5.2.1 Multi-stream state . . . 4
  - 5.2.2 Connection maps (dimensions) . . . 4
  - 5.2.3 Update rule (interpretable form) . . . 4

6 mHC vs mHC-lite: enforcing stability **4**
- 6.1 HC (unconstrained multi-stream baseline) . . . 4
- 6.2 mHC (Sinkhorn projection) . . . 4
- 6.3 mHC-lite (exact by construction) . . . 4
- 6.4 Scaling issue and approximation . . . 5

7 Experimental protocol **5**
- 7.1 Ablation axes . . . 5
- 7.2 Inductive stage (graph family generalization) . . . 5

8 Reproducibility and reporting **5**

9 Expected outcomes **5**

10 Repository layout (minimal) **5**

1 Research question and scope

1.1 Central question

RQ. *Can deep GNNs predict betweenness centrality in an inductive setting (train on small graphs, test on larger graphs), and do multi-stream residual mechanisms (HC/mHC/mHC-lite) improve depth scaling and Top-K retrieval compared to strong deep-GNN baselines?*

1.2 What "success" means

We consider BC prediction successful if the model: (i) ranks nodes correctly (global correlation), and (ii) reliably retrieves the highest-BC nodes (Top-K).

1.3 Inductive-only experimental protocol

We train on many small graphs and test on larger graphs drawn from multiple canonical families (e.g., ER/BA/SBM/WS/RGG), to avoid learning a single "graph style". We explicitly report in-distribution (ID) vs out-of-distribution (OOD) generalization: ID uses the same families for train/test, while OOD uses disjoint families (e.g., train ER+BA+SBM, test WS+RGG).

Why HC, and why constraints. HC is the baseline multi-stream routing mechanism that may help depth scaling on its own. mHC/mHC-lite are constrained variants: they keep the same routing but constrain H^{res} to test whether stability, not just routing capacity, is the key factor.

2 Main hypotheses

H1 (depth bottleneck). Increasing depth in standard MPNNs eventually degrades performance due to over-smoothing/over-squashing and optimization issues.

H2 (multi-stream routing helps depth). HC-style multi-stream residual routing improves optimization stability and preserves useful signal across depth, enabling deeper networks to outperform strong deep-GNN baselines on ranking metrics.

H3 (structure helps). Injecting structural information (e.g., LapPE, degree features) improves BC ranking and may reduce the required depth for comparable performance.

H4 (constraint matters at extreme depth). At sufficient depth, enforcing (approx./exact) doubly-stochastic constraints on H^{res} (mHC/mHC-lite) improves stability and ranking performance compared to unconstrained HC, while differences may be smaller at shallow depth.

3 Task definition: labels, objectives, metrics

3.1 Datasets (inductive protocol)

Train set: $M_{mathrmtrain}$ graphs of size $N_{mathrmtrain}$ (e.g., 100 graphs of 100 nodes). Test set: graphs of size $N_{mathrmtest}$ (e.g., 1000 nodes), ideally multiple graphs to estimate variance. Families: at least five standard families from the literature: ER (Erdős–Rényi), BA (Barabási–Albert), SBM, WS, and RGG (or configuration model). We generate multiple families explicitly to prevent the model from learning a single graph "style".

3.2 Betweenness centrality labels

For small graphs, BC is computed exactly (Brandes algorithm). For larger graphs, labels may be computed either exactly (if feasible, e.g., 1000 nodes depending on density) or approximately (sampling-based BC), as an engineering trade-off.

3.3 Target transformation

BC is heavy-tailed. We use:

$$y(v) = \log(1 + \mathrm{BC}(v)), \qquad \tilde{y}(v) = \frac{y(v) - \mu_{\text{train}}}{\sigma_{\text{train}}}.$$

3.4 Training objectives

We optimize a ranking objective only, since Top-K retrieval is the primary goal.

Ranking objective. Pairwise RankNet-style loss over sampled node pairs (u, v):

$$\mathcal{L}_{\text{rank}} = \sum_{(u,v)} \log\big(1 + \exp\big(- s_{uv}(\hat{y}(u) - \hat{y}(v))\big)\big)\,, \quad s_{uv} = \mathrm{sign}(y(u) - y(v)).$$

3.5 Evaluation metrics

We report:

Global ranking agreement. Spearman ρ (and optionally Kendall τ) between ranks($\hat{y}$) and ranks(y).

Top-K retrieval. Precision@K and NDCG@K for $K \in \{10, 20, 50\}$ and proportional K (top 1%, top 5%).

4 Inputs: features and structural encodings

4.1 Feature sets

We define three input configurations tailored to synthetic graphs:

1. **Option 0 (structural-only):** degree, $\log(1 + \text{degree})$, and LapPE (k eigenvectors).
2. **Option 1 (random features):** X
   sim
   $mathcalN(0, I)$ or one-hot IDs (noting that one-hot can be risky inductively).
3. **Option 2 (no node features):** structure-only learning via message passing (often very relevant for BC).

For LapPE, we use the normalized Laplacian:

$$L = I - D^{-1/2}AD^{-1/2}, \quad LU = ULambda, \quad X' = [X; Vert; U_k],$$

and apply random sign flips during training to handle eigenvector sign ambiguity.

5 Models to compare

5.1 Baselines

Non-GNN baselines. Constant predictor, MLP on X, and MLP on X+structure.

Shallow GNN baselines. GCN / GraphSAGE / GAT / GIN with $L \in \{2, 3\}$ (common practice).

Deep GNN baselines (strong references). Representative depth-stabilized architectures (e.g., GCNII, APPNP/GPRGNN, JKNet), used to ensure comparisons are meaningful when L is large. Our focus is on depth scaling and Top-K recovery, which are critical for BC.

5.2 Proposed: multi-stream routing in GNNs

Variants. We explicitly compare three routing variants: **HC-GNN (unconstrained)**, **mHC-GNN (Sinkhorn)**, and **mHC-lite-GNN (convex combination of permutations)**.

5.2.1 Multi-stream state

Each node i carries n parallel streams at layer l:

$$x_i^{(l)} \in \mathbb{R}^{n \times d}.$$

5.2.2 Connection maps (dimensions)

$$H_{l,i}^{\text{pre}} \in \mathbb{R}^{1 \times n}, \qquad H_{l,i}^{\text{post}} \in \mathbb{R}^{1 \times n}, \qquad H_{l,i}^{\text{res}} \in \mathbb{R}^{n \times n}.$$

5.2.3 Update rule (interpretable form)

$$\bar{x}_i^{(l)} = H_{l,i}^{\text{pre}} x_i^{(l)} \in \mathbb{R}^{1 \times d},$$

$$m_i^{(l)} = F_{\text{GNN}}\Big(\bar{x}_i^{(l)}, \{x_j^{(l)} : j \in \mathcal{N}_i\}\Big) \in \mathbb{R}^{1 \times d},$$

$$x_i^{(l+1)} = H_{l,i}^{\text{res}} x_i^{(l)} + (H_{l,i}^{\text{post}})^\top m_i^{(l)}.$$

How H^{pre}, H^{post}, H^{res} are built. Static HC: H are learned per-layer global parameters shared across nodes. **Dynamic HC (optional):** H depend on $x_i^{(l)}$ via a small MLP + nonlinearity (sigmoid/softmax), often after normalization; we start with static-only for robustness and optionally add dynamic components later (static + dynamic is closest to the paper's recommended setting).

Compute intuition. The expensive backbone F_{GNN} operates on $\bar{x}_i^{(l)} \in \mathbb{R}^d$ (not $\mathbb{R}^{nd}$). Overhead mostly comes from multi-stream activations and the small $n \times n$ residual mixing.

6 mHC vs mHC-lite: enforcing stability

6.1 HC (unconstrained multi-stream baseline)

We include **HC-GNN** as the direct multi-stream baseline, using the same update rule but *without* enforcing any constraint on $H_{l,i}^{\text{res}}$. This isolates the effect of the manifold constraint: HC tests whether routing alone helps depth, while mHC/mHC-lite test whether *constraining* the residual mixing is necessary for stable deep training.

6.2 mHC (Sinkhorn projection)

mHC constrains $H_{l,i}^{\text{res}}$ to be approximately doubly-stochastic ($\in B_n$) by: (i) building a positive matrix (via exponentiation) and (ii) applying T Sinkhorn-Knopp iterations (row/column normalization).

6.3 mHC-lite (exact by construction)

Using Birkhoff–von Neumann:

$$H^{\text{res}} = \sum_{k=1}^{n!} a_k P^{(k)}, \qquad a = \text{softmax}(z),$$

so $a_k \geq 0$ and $\sum_k a_k = 1$ guarantee a convex combination of permutation matrices, hence exact doubly-stochasticity.

6.4 Scaling issue and approximation

Since $n!$ grows quickly ($4! = 24$, $10! = 3{,}628{,}800$), we consider approximations for larger n: selecting a subset of permutations (Top-M or stochastic sampling) and re-normalizing weights, while preserving convex-combination form.

7 Experimental protocol

7.1 Ablation axes

We vary:

$$L \in \{2, 4, 8, 16, 32\}, \quad n \in \{2, 4\}, \quad \text{inputs} \in \{\text{struct-only}, \text{random}, \text{none}\}.$$

Backbone order: start with GCN (fast, stable), then expand to SAGE/GAT/GIN.

7.2 Inductive stage (graph family generalization)

Train on many small graphs (e.g., 100 graphs of 100 nodes), test on larger graphs (e.g., multiple graphs of 1000 nodes), across multiple canonical families: ER, BA, SBM, WS, RGG (at least five). We report both ID (train families = test families) and OOD (train families $\neq$ test families) settings to test transfer beyond family-specific style.

8 Reproducibility and reporting

We use fixed random seeds, save configs and checkpoints, and report: Spearman/NDCG/Precision@K, training curves, runtime and (if possible) memory footprint.

9 Expected outcomes

We expect that naive depth degrades performance without explicit routing mechanisms, that HC already improves depth scaling, that mHC/mHC-lite become important as L grows (stability/variance/collapse), and that Top-K retrieval improves on larger graphs.

10 Repository layout (minimal)

- `configs/` : YAML configs (dataset/model/training/metrics)
- `src/data/` : loaders, splits, BC label generation, LapPE
- `src/models/` : baselines, deep baselines, HC/mHC/mHC-lite wrappers
- `src/train.py` : training loop + logging
- `src/eval.py` : ranking and Top-K evaluation
- `src/utils/` : seeds, checkpoints, logging